

Домашнее задание 08 Гареевой Фариды

Проекции и материализованные представления

Цель:

понять, как работают проекции и материализованные представления в ClickHouse;

научиться создавать и использовать их для оптимизации запросов;

Описание/Пошаговая инструкция выполнения домашнего задания:

Для выполнения задания следуйте следующим шагам:

Создание таблицы:

Создайте таблицу **sales** с полями:

id (UInt32) — уникальный идентификатор продажи

product_id (UInt32) — идентификатор продукта

quantity (UInt32) — количество проданных единиц

price (Float32) — цена за единицу

sale_date (DateTime) — дата продажи

Заполните таблицу тестовыми данными.

```
CREATE database hw08;

--создаем таблицу
create table if not exists hw08.sales (
  id UInt32,
  product_id UInt32,
  quantity UInt32,
  price Float32,
  sale_date DateTime
) engine = MergeTree
order by (product_id)
;

--заполняем данными
insert into hw08.sales (id, product_id, quantity, price, sale_date)
select number,
rand()%10000,
rand()%1005,
rand()%10031,
toDate(toInt32(today()) - (rand() % 730))
from numbers(1,100000000);
```

Создание проекции:

Создайте проекцию для таблицы `sales`, которая будет агрегировать данные по `product_id` и считать общую сумму продаж (количество и сумма по цене) за каждый продукт.

```
--создаем проекцию для таблицы:
ALTER TABLE hw08.sales ADD projection sales_projection2 (select product_id,
sum(price*quantity) as summ
group by product_id
);
--заполняем ее данными:
ALTER TABLE hw08.sales MATERIALIZE projection sales_projection2;
```

Создание материализованного представления:

Создайте материализованное представление `sales_mv`, которое будет автоматически обновляться при вставке новых данных в таблицу `sales`. Оно должно хранить общие продажи по продуктам с полями:

`product_id`

`total_quantity`

`total_sales`

```
--создаем таблицу, в которую будем лить данные
create table hw08.sales_agg (
product_id UInt32,
total_quantity UInt32,
total_sales Float32) engine = SummingMergeTree
order by (product_id);

--создаем материализованное представление
create materialized view sales_mv to hw08.sales_agg
as select product_id,
sum(quantity) as total_quantity,
sum(quantity*price) as total_sales
from hw08.sales
group by product_id;

--заливаем данные в hw08.sales_agg
insert into hw08.sales_agg
select product_id,
sum(quantity) as total_quantity,
sum(quantity*price) as total_sales
from hw08.sales
group by product_id;
```

Запросы к данным:

Напишите запрос, который извлекает данные из проекции `sales_projection`.

```
select product_id,
sum(price*quantity) as sum
from hw08.sales
group by product_id;
```

Напишите запрос, который извлекает данные из материализованного представления `sales_mv`.

```
select product_id,
sum(total_quantity) ,
sum(total_sales)
from sales_mv
group by product_id;
```

Сравнение производительности:

Сравните время выполнения запроса к основной таблице sales с запросом к проекции sales_projection и материализованному представлению sales_mv. Обратите внимание на разницу в производительности.

Запросы выполняем в clickhouse-client, чтобы видеть количество обработанных строк и объем используемой памяти.

```
--Запрос к проекции
select product_id,
sum(price*quantity) as sum
from hw08.sales
group by product_id;
10001 rows in set. Elapsed: 0.017 sec. Processed 30.00 thousand rows, 600.02 KB (1.80 million
rows/s., 36.09 MB/s.)
Peak memory usage: 2.30 MiB.
```

Чтобы убедиться, что запрос выполнялся именно к проекции, выполним запрос с explain

```
clickhouse-server-1 :) explain select product_id,
sum(price*quantity) as sum
from hw08.sales
group by product_id;
```

```
EXPLAIN
SELECT
    product_id,
    sum(price * quantity) AS sum
FROM hw08.sales
GROUP BY product_id
```

```
Query id: 4ee9bf2b-6c71-4996-a57e-2ce8cf03c521
```

```
explain
1. Expression ((Project names + Projection))
2.   Aggregating
3.     Expression
4.       ReadFromMergeTree (sales_projection2)
```

```
4 rows in set. Elapsed: 0.004 sec.
```

Выполним запрос к материализованному представлению в клиенте

```
select product_id,
sum(total_quantity) ,
sum(total_sales)
from sales_mv
group by product_id;
```

10001 rows in set. Elapsed: **0.011 sec.** Processed **10.00 thousand rows**, **120.02 KB** (**878.79** thousand rows/s., **10.55** MB/s.)

Peak memory usage: **309.04 KiB.**

Для выполнения запроса к самой исходной таблице hw08.sales нужно либо отключить использование проекций, либо удалить проекцию. Переопределить значение настройки `optimize_use_projections`, которая отключает использование проекций почему-то не получилось. Пыталась создать пользовательский конфигурационный файл /etc/clickhouse-server/config.d/config.xml и после перезапуска clickhouse-server, но почему-то новая настройки не применилась.

```
<clickhouse>
<settings>
<optimize_use_projections>0</optimize_use_projections>
</settings>
</clickhouse>
```

После перезапуска сервера настройка не меняется:

```
select value from system.settings where name = 'optimize_use_projections';
```

Поэтому пришлось удалить проекцию:

```
ALTER TABLE hw08.sales drop projection sales_projection2;
```

После удаления проекции выполняем запрос:

```
Explain select product_id,
sum(price*quantity) as sum
from hw08.sales
group by product_id;
```

Видно, что теперь запрос будет по исходной таблице hw08.sales:

```
clickhouse-server-1 :) Explain select product_id,
sum(price*quantity) as sum
from hw08.sales
group by product_id;
```

```
EXPLAIN
SELECT
    product_id,
    sum(price * quantity) AS sum
FROM hw08.sales
GROUP BY product_id
```

Query id: 10d8f59f-05b8-4f46-8356-da3500113b5b

```
explain
1. Expression ((Project names + Projection))
2. Aggregating
3. Expression ((Before GROUP BY + Change column names to column identifiers))
4. ReadFromMergeTree (hw08.sales)
```

```
select product_id,
sum(price*quantity) as sum
from hw08.sales
group by product_id;
```

10001 rows in set. Elapsed: **0.547 sec.** Processed **100.00 million** rows, 1.20 GB (182.87 million rows/s., 2.19 GB/s.)
Peak memory usage: **9.38 MiB.**

Сравнительная таблица производительности

	Elapsed, sec	Processed, rows	Memory total	Speed rows/sec	Speed memory/sec	Peak memory usage
Таблица	0.547	100.00 million	1.20 GB	182.87 million	2.19 GB/s.	9.38 MiB
Проекция	0.017	30.00 thousand	600.02 KB	1.80 million	36.09 MB/s	2.30 MiB
MV	0.011	10.00 thousand	120.02 KB	878.79 thousand	10.55 MB/s.	309.04 KiB

Самой выгодной для агрегирующего запроса оказалось материализованное представление.